Rust Tuesday

Build it. Strip it. Explain it.

A two-hour workshop on Rust and reverse engineering

Start with a small algorithm. Turn it into a command-line program. Compile two versions. Then investigate the executable with REA and explain what the machine is doing, using evidence from the binary.

The route

```
Two Sum -> functions -> trait + impl -> native binary
                                            |
reconstructed behavior <- evidence <- REA <---+
```

What you will make

- A pair-finder CLI with two interchangeable algorithms.
- Tests for matches, missing answers, duplicates and edge cases.
- A debug binary and an optimized, stripped challenge binary.
- A short investigation and a behaviorally compatible reconstruction.

Who this is for

Programmers who know variables, loops and basic terminal commands. Rust experience and assembly knowledge are optional. Work in pairs and swap keyboard roles halfway. One machine per pair needs Rust; the native REA session can run on a shared facilitator machine if it is prepared in advance.

Success means you can explain the Rust you wrote, trace one behavior into the binary, and say which conclusions the evidence supports. Exact source recovery is not an outcome promised by this workshop.

Public materials: github.com/superposition/rust-tuesday

Prepared 15 October 2026. Duration and public repository confirmed; event date TBD. This guide is ready to rehearse. Native REA provider execution is still pending.

01 / Prepare the session

The plan before the plan goes live

At least one day ahead - facilitator, 45-60 minutes

1. Run the reference and tests. Create a learner directory and confirm its TODOs are the only intended failures.
2. Configure REA plus a native analysis provider. Check the exact host and CPU used in the event; macOS and Linux executables are not interchangeable.
3. Import the debug binary and find one algorithm function. Repeat a focused inspection on the stripped binary. Save real output and its binary hash.
4. Rehearse the 15-minute guided inspection and 20-minute reconstruction. Capture a short fallback walkthrough from that successful run.
5. Freeze tool versions and the repository commit. Send the setup instructions, and keep a working instructor machine ready for anyone with setup trouble.

Participant preflight

```
git clone https://github.com/superposition/rust-tuesday.git
cd rust-tuesday
rustc --version
cargo test --locked
bash scripts/new-learner.sh ../rust-tuesday-learner
cd ../rust-tuesday-learner
cargo check --locked
```

The learner project compiles. Its tests initially panic at TODOs. The facilitator reference already passes; do not open its solution while implementing the exercise.

Tool choices

Rust 1.74+; a terminal and editor; Python 3 for packaging and comparison. REA 6.5.0 is the published version checked for this draft. Use a supported Node version and a working Ghidra or Hopper installation before the session (page 8). For this guide, use macOS or Linux; Windows users can use a rehearsed WSL setup.

Go/no-go for the native segment

A provider health check is necessary but insufficient. The facilitator must have opened this binary, obtained a relevant function, and saved evidence. This repo does not yet contain that provider capture. If rehearsal fails, announce a changed agenda or use the basic binary tools on page 9; do not call that a completed REA lab.

02 / The two-hour run of show

Eight blocks, with visible checkpoints

00-10 / Start with behavior

Run `pair-finder a 9 2 7 11 15`. Predict `0 1`. Draw the two indices and agree that the same array element cannot be used twice. Show the session route.

10-25 / Read the Rust vocabulary

Walk through `fn`, references, a slice, `Option`, a trait and two types. Contrast `impl Request` with `impl PairSearch for BruteForce`. Checkpoint: each pair points to an associated function, a method and a free function in the supplied CLI.

25-45 / Build the loop solution

Implement `BruteForce::find` and `solve` in the learner project. Test a normal match, duplicates and no match. Checkpoint: explain why `i < j` matters.

45-65 / Add the hash implementation

Swap drivers. Implement `HashLookup::find` with the same contract. Preserve the earliest index and check before insertion. Run the whole test suite. Checkpoint: changing the implementation does not change the caller's required interface.

65-75 / Compile and predict

Package debug and release binaries from the reference. Compare sizes, format and symbols. Write one prediction about what optimization and stripping will obscure.

75-90 / Guided REA inspection

Facilitator opens the warmup binary. Find a relevant function, inspect pseudocode or instructions, and follow one reference. Record a claim with a location.

90-110 / Reconstruct from a binary

Use the stripped executable and brief in a separate directory. Probe behavior, write a compatible CLI, then run the comparison script. A simple implementation of the observed contract is enough; native algorithm attribution needs more evidence.

110-120 / Reveal and explain

Compare source, guesses and evidence. Each pair shares one correct inference, one uncertainty and an exit ticket. Optional `dyn Trait` discussion only if ahead.

If behind at minute 45, walk through the reference hash solution. Keep the native inspection and ten-minute debrief. A focused, explained program is the priority.

03 / Pick one problem

Two Sum is the core; the others are extensions

Core: Two Sum, LeetCode #1

Our original workshop formulation: given a target and a list of signed integers, find two different positions whose values add to the target. Return the indices, not the values. The CLI supplies all inputs at runtime.

Our contract extends the familiar exercise

- Zero-based indices, always `i < j`.
- If several pairs work, choose the smallest `j`, then the smallest `i`.
- No answer, an empty list, or one item returns `None` internally and `none` on stdout.
- Numbers and target must fit `i32`. Use `i64` for sum/complement arithmetic so an overflow cannot become a false match.
- Valid runs exit 0. Invalid arguments write to stderr and exit 2.

```
mode target numbers            stdout
a    9      2 7 11 15          0 1
b    6      3 3                0 1
a    6      3                  none
b    0      -3 4 3 90          0 2
a    5      1 2 3 4            1 2
b    5      1 1 4              0 2
a    0      (empty list)       none
```

Why two implementations?

Nested loops are easy to follow and take O(n squared) time in the worst case, with O(1) auxiliary space. A hash map trades O(n) extra space for expected O(n) time under ordinary hash-table assumptions. Tiny examples and CLI startup time do not establish performance; structural evidence matters in the binary lab.

Optional next problems

Contains Duplicate (#217): return a boolean, compare a loop with a set, discuss ownership and `HashSet`. Valid Parentheses (#20): use a stack and `match`, return a validity result, inspect push/pop branches. Do one in a later session; both are outside the two-hour core. Problem links appear on page 12.

Before coding: sketch the loop for `[3, 3]`, target 6. Explain why `[3]` is not a match, and which pair wins for `[1, 2, 3, 4]`, target 5.

04 / Read the Rust

One behavior, two concrete types

```
pub trait PairSearch {
    fn find(&self, nums: &[i32], target: i32)
        -> Option<(usize, usize)>;
}

pub struct BruteForce;
pub struct HashLookup;

pub fn solve<S: PairSearch>(
    solver: &S, nums: &[i32], target: i32,
) -> Option<(usize, usize)> {
    solver.find(nums, target)
}
```

`fn` **defines a function.** Its parameter and return types describe the interface. An expression at the end of a block can supply the return value.

`&[i32]` **borrows a slice.** The caller keeps ownership of its vector. The search reads the elements without taking the vector or cloning it.

`Option<(usize, usize)>` **expresses absence.** `Some((i, j))` is a pair of indices; `None` means no pair. A tuple groups the two values. There is no made-up sentinel index.

A trait describes shared behavior. `PairSearch` requires a `find` method. `impl PairSearch for BruteForce` supplies that behavior for one concrete type; the hash implementation satisfies the same requirement.

An inherent `impl` **belongs to a type.** In `src/main.rs`, `impl Request` contains `Request::parse(...)`, an associated function without `self`, and `request.run()`, an inherent method with `&self`. `parse_number(...)` is a free function.

The generic bound is a compile-time requirement. `S: PairSearch` says which behavior a concrete `S` must provide. The compiler specializes generic code for used concrete types; optimization may inline or merge it. This code does not use `dyn PairSearch`, so the presence of a trait does not imply a runtime vtable.

Checkpoint: explain the difference between a trait declaration, its implementation, and the call through `solve`. Then point to where `Result` handles parse failure.

05 / Build the program

Keep the CLI; implement the search

Work in the learner directory. `src/main.rs` is supplied. Implement only the three TODOs in `src/lib.rs`; that keeps parsing from taking over the workshop.

Step A / delegate

Complete `solve` by calling the trait method on `solver`. Use `cargo check --locked` to check your types. A successful check does not mean the TODOs are solved.

Step B / nested loops

Scan `j` from left to right. For each `j`, examine earlier indices `i` from left to right. Widen both values before addition. Return the first pair that matches, then return `None` if all candidates fail. The order of loops implements our tie-break.

```
cargo test --locked --test contract examples_and_boundaries
```

The shared test still fails when it reaches the unfinished hash implementation. Use `cargo run -- a 9 2 7 11 15` to exercise the loop implementation on its own.

Step C / hash lookup

Keep a map from previously seen values to their earliest indices. For each `(j, value)`, compute the needed complement as `i64`. Look it up first. Only if there is no match should you insert the current value; keep an existing index.

```
// Useful ingredients; assemble them into the algorithm.
use std::collections::HashMap;
let mut seen = HashMap::new();
let needed = i64::from(target) - i64::from(value);
seen.get(&needed);
seen.entry(i64::from(value)).or_insert(j);
```

This ingredient snippet is not a complete program. Discuss the map's inferred key/value types and why `if let Some(&i)` copies an index from a borrowed entry.

```
cargo fmt
cargo test --locked
cargo run -- b 6 3 3
```

Before moving on, both modes must agree. Add or explain a case involving a negative number and a case near the `i32` boundary. The supplied exhaustive small-domain test compares both implementations on 12,285 input/target combinations.

06 / Compile two views

Keep a readable warmup and a harder challenge

Return to the facilitator reference repository. Package native binaries:

```
bash scripts/package-binaries.sh
./artifacts/warmup/pair-finder a 9 2 7 11 15
./artifacts/challenge/pair-finder b 9 2 7 11 15
file artifacts/warmup/pair-finder
file artifacts/challenge/pair-finder
```

The script builds for the current Rust host triple. It records compiler details, binary sizes and SHA-256 hashes in `artifacts/manifest.json`. The first package is preserved; pass a fresh destination argument when rebuilding.

Warmup / development profile

`opt-level = 0`, `debug = 2`, `strip = "none"`, `split-debuginfo = "off"`. Symbol names and debug information give the group a route into unfamiliar output. The packaged executable carries debug information rather than relying on a separate macOS build-directory sidecar. What an analysis provider displays still varies.

Challenge / release profile

`opt-level = 3`, `debug = 0`, `strip = "symbols"`. Optimization can inline functions, remove branches and rearrange logic. Stripping removes symbol/debug information according to the target toolchain; it does not remove every string or import. Both profiles keep overflow checks enabled; the algorithm widens arithmetic anyway.

Observe before you infer

```
nm artifacts/warmup/pair-finder > /tmp/warmup-symbols.txt
nm artifacts/challenge/pair-finder > /tmp/challenge-symbols.txt
strings artifacts/challenge/pair-finder > /tmp/challenge-strings.txt
```

`nm` may report no symbols or retain some dynamic names. Rust runtime and library code will dominate much of the output. Look for your program's behavior, not a one-to-one translation of every source line.

Handoff rule

Give the investigator only the challenge directory and its binary hash. Start a fresh agent conversation with the supplied brief. Keep the source closed. This is a classroom constraint, not a secure sandbox: a public answer can still be fetched.

Predict which names survive, then record what you actually see.

07 / Get REA ready

Do this before the clock starts

REA coordinates analysis tools. For native function inspection it needs a working provider. Its Rust dependency-metadata command is a different feature; ordinary Cargo output may contain no cargo-auditable metadata, and metadata is not pseudocode.

Version baseline checked for this draft

REA 6.5.0; supported Node ranges are 22.19+, within 22.x, or 24.11+, within 24.x, or 26+. The checked Ghidra route expects 12.1.x; its documented verified pairing is Ghidra 12.1.4 and JDK 21. Use the JDK version and architecture accepted by the actual Ghidra installation. Recheck the pinned setup docs before the event.

Install Ghidra and its full JDK using their official distribution instructions. Set `GHIDRA_INSTALL_DIR` to the extracted installation's absolute path and `JAVA_HOME` to the matching JDK home. The native decompiler for your host must exist. REA does not install Ghidra or Java for you. Hopper is an alternative if already installed and usable under its license.

```
# Run after installing a supported Node runtime.
npm install --global rea-agents@6.5.0
rea --version
rea providers --json
rea doctor --provider ghidra --json
```

For agent integration, run `rea setup`, select the intended agent, review the changes, complete setup, then restart that agent. The CLI route also works without agent registration. If using Hopper, substitute `hopper` for the provider name.

The real acceptance check

Run `rea analyze` on this workshop's warmup binary, then retrieve a relevant function. Save the output and any warnings. Do not treat a provider listing or a successful exit as proof that the relevant algorithm was recovered.

Current rehearsal status

The authoring machine has Rust 1.90.0 and Node 22.14.0. That Node version is below the REA baseline. No REA CLI or Ghidra/Hopper installation was found in the checked locations. Native analysis was not run. The preparation task is to install or use a suitable prepared machine and capture the actual session before teaching.

The commands on the next page were checked against upstream documentation; provider-specific output and timing remain unverified locally.

08 / Investigate with evidence

Begin with one behavior and one function

From the repository root, with a working Ghidra provider:

```
binary="$(pwd)/artifacts/warmup/pair-finder"
mkdir -p tmp/rea
rea analyze "$binary" --provider ghidra --json \
  > tmp/rea/overview.json
rea search "$binary" "find" --kind procedures \
  --provider ghidra --json > tmp/rea/search.json
```

Read the returned inventory. If the search finds nothing, try `BruteForce`, `HashLookup`, or inspect the overview. Copy an exact returned function name or address into the following prompt; the address is not portable between builds.

```
printf 'Paste an observed procedure name/address: '
IFS= read -r procedure
rea function "$binary" "$procedure" --provider ghidra \
  --json > tmp/rea/function.json
rea decompile "$binary" "$procedure" --provider ghidra \
  --json > tmp/rea/decompile.json
rea xrefs "$binary" "$procedure" --provider ghidra \
  --json > tmp/rea/xrefs.json
```

Ask narrowly: where are two input elements compared against the target? What branch returns a result? Which call path invokes it? What evidence suggests a lookup table in the other mode? Ignore unrelated startup and formatting routines.

Repeat with the challenge binary. Named searches may fail after stripping. Start from observed input/output strings and their references, then follow the control flow. Do not invent a function boundary or describe a guess as recovered Rust.

Fallback when the provider fails

Use the facilitator's captured evidence for the exact hash, if prepared. Otherwise inspect the binary with `file`, `strings`, `nm` and `objdump -d` (`otool -tvV` on macOS is another option). Save the assembly and connect a small part to behavior. This supports the binary lesson but does not complete the planned REA segment.

Every claim needs: artifact/hash; command or address; observed output; interpretation; uncertainty. Pseudocode is a tool's reconstruction, not source truth.

09 / Rebuild the behavior

A 20-minute binary-only challenge

Use `docs/BINARY-BRIEF.md` or the copy packaged beside the stripped executable. Start a fresh project and agent conversation outside the source checkout.

Minutes 0-5 / probe

Try both modes, a valid pair, no pair, duplicate values, negative values, empty input, one item, and multiple possible answers. Record stdout and exit status. Add an arithmetic boundary case. Ask what these observations do not tell you.

Minutes 5-15 / reconstruct

Use observed behavior and focused REA evidence to write a compatible CLI. A simple loop implementation for both modes can satisfy the observable contract; claiming that the original uses a hash map requires structural evidence from analysis.

Minutes 15-20 / compare

Compile your reconstruction as a native executable for the same machine, then run this from the workshop repository, substituting its actual path:

```
python3 scripts/compare-binaries.py \
  artifacts/challenge/pair-finder \
  /absolute/path/to/your/rebuilt-program
```

The script runs 225 cases across the two modes and invalid inputs. It compares exit status, stdout, and whether stderr is present. Exact error wording is not part of the contract. A mismatch is a useful clue: reduce it to a smaller example.

Passing means the tested behaviors agree. It does not prove all inputs, equivalent performance, identical instructions, the original names, or the same source design.

Prompt for the agent

Investigate only the provided binary and brief. Do not read or fetch workshop source. Record the hash and platform. Separate observation, inference and uncertainty. Use the configured REA native provider, cite exact locations, and test your reconstruction against the original. Report unavailable evidence explicitly.

Reveal only after comparison

Open the reference source. Which observations were right? Which details came from debug symbols? What did optimization obscure? Could two different Rust programs produce the behavior you recovered? Keep that distinction in the final report.

10 / Worksheet and debrief

Hand in evidence, not just an answer

Names / date: _____

Binary SHA-256 / host: _____

REA / provider / compiler versions: _____

Claim 1 / input and output

Command and observed output: _____

Inference and a counterexample to try: _____

Claim 2 / algorithm

Function address or symbol and observed instructions: _____

What this supports; what remains unknown: _____

Claim 3 / compilation

One difference between warmup and challenge: _____

Evidence; alternative explanation: _____

Reconstruction result

Cases passed / first mismatch: _____

One unresolved question: _____

Exit ticket / say it in your own words

- What does `impl PairSearch for HashLookup` promise?
- Why can the same `solve` function call either implementation?
- Who owns the vector when `find` borrows a slice?
- Why is a trait declaration not proof of a vtable in this binary?
- What did you recover, and what would be overclaiming?

Facilitator rubric: 1 point each for correct behavior, clear Rust explanation, three evidence-backed claims, explicit uncertainty, and a comparison result. All five are the target; the unresolved question is a useful outcome, not a penalty.

11 / References and next steps

Keep the materials editable

The repository is the working plan. Edit `docs/WORKSHOP.md`, rebuild the PDF with `scripts/build-pdf.py`, and visually review the rendered pages before sharing. `docs/PREPARATION.md` tracks rehearsal work. `docs/VERIFICATION.md` distinguishes executed checks from the planned native tool session.

Primary references / checked 15 October 2026

- Two Sum, LeetCode #1 - the core problem family. Our contract and examples are independently written and include extra cases.
- Contains Duplicate, LeetCode #217 and Valid Parentheses, LeetCode #20 are follow-up exercises.
- Rust Book: traits, methods, and borrowing provide language background for the source lesson.
- Cargo profiles - debug, optimization, stripping and split-debug settings.
- REA repository and npm package - project and release.
- REA setup, CLI, and native investigation form the pinned documentation snapshot used for this draft's commands.

Stretch: dispatch

After the core session, write a second caller accepting `&dyn PairSearch`. Explain static versus dynamic dispatch, then compare emitted code. Optimization can devirtualize calls; an indirect call is evidence to investigate, not a universal guarantee that a particular source construct survives.

Stretch: a different problem

Reuse the method: state a contract, make tests, write two approaches where useful, compile, inspect, reconstruct and compare. A trait should express a useful shared behavior; it does not need to appear in every small program.

Before announcing the event

Complete the native provider rehearsal, choose the date and venue, and distribute the setup instructions. Keep the generated binaries matched to each participant's platform and retain their hashes with all saved evidence.